

Air University



SSDD (Lab)

Year 2026

Mr. Anjum Rehman

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: March 2, 2026

Quiz # 01

Question 01

A child Accidentally enters the wrong password multiple times.

1. Is this an error or failure?

Ans: Technically, it starts as an **error** (a human action that produces an incorrect result). However, once the system denies access because the input doesn't match the database, it is a **failure** of that specific login attempt to perform its intended function.

2. What state should the system move to after multiple failure attempts?

Ans: After multiple failed attempts, the system should move to a **Lockout** or **Restricted State**.

- **Temporary Lockout:** Disabling access for a specific duration (e.g., 30 minutes).
- **Progressive Delay:** Increasing the wait time between each subsequent attempt (Throttling).
- **Administrative Intervention:** Requiring a password reset or identity verification.

3. What preventive controls can be applied?

Ans:

- **Input Validation:** Ensuring the password meets format requirements before processing.
- **CAPTCHA:** To ensure the "child" isn't a brute-force bot.
- **Multi-Factor Authentication (MFA):** Preventing access even if the password is eventually guessed.
- **Visual Cues:** Showing "Caps Lock is ON" to prevent accidental typos.

Question 02

1. Differentiate between state, transition, and event.

Ans:

Term	Definition	Example
State	A condition or situation during the life of an object.	"Logged Out" or "Authenticated"
Event	A significant occurrence that triggers a reaction.	User clicks the "Submit" button.
Transition	The movement from one state to another.	Moving from "Locked" to "Unlocked."

2. Explain how a misuse case can differ from a vulnerability

Ans:

- **Misuse Case:** Describes a scenario from the perspective of an **actor** (attacker) trying to intentionally harm the system (e.g., "An attacker steals a session cookie"). It focuses on the **behavior**.
- **Vulnerability:** A **weakness** or flaw in the system's design, implementation, or operation (e.g., "The system doesn't encrypt session cookies"). It is the **hole** that allows the misuse case to happen.

3. Explain the formula: $\text{Risk} = \text{threat} \times \text{vulnerability}$. Is this formula always sufficient? Why or why not?

Ans: The standard qualitative formula is:

$$\text{Risk} = \text{Threat} \times \text{Vulnerability}$$

Is it always sufficient? No. This formula is a great starting point, but it lacks **Impact**. A high-threat, high-vulnerability event on a non-critical system (like a public cafeteria menu) is lower risk than the same event on a core banking database. A more complete version is:

$$\text{Risk} = \text{Threat} \times \text{Vulnerability} \times \text{Impact}$$

4. Differentiate between error, fault, defect, bug, and failure with examples.

Ans:

Term	Definition	Example
Error	A human action or mental mistake made by a programmer, designer, or tester that produces an incorrect result.	A developer mistakenly believes that the month of February always has 28 days while writing a calendar function.
Fault / Defect	The physical manifestation of the error in the software (the actual "flaw" sitting in the code or documentation).	The code contains a hardcoded condition: if (month == 2 && day > 28) {return "Invalid Date"; }.
Bug	An informal term used to describe a fault, defect, or error. It is the general "catch-all" label used in the industry.	A tester notes in a report: "There is a bug in the date validation logic for leap years."
Failure	The external behavior observed when the system does not perform its required function during execution.	On February 29th, 2024, a user tries to book a flight, but the app blocks them and says "Invalid Date," preventing the sale.

5. How can misuse cases improve system resilience?

Ans: Misuse cases force developers to think like an adversary. They improve resilience by:

- Defining what the system **should not** do.
- Designing security controls (like rate-limiting or encryption) during the architecture phase rather than "bolting them on" later.
- Providing a roadmap for security teams to perform penetration testing.

Question 03

An attacker clones an employee's keycard and attempts entry multiple times.

1. Identify the threat.

Ans: The threat here is **Unauthorized Physical Access** via **Identity Spoofing**. The specific threat actor is an external intruder (the attacker) using a cloned credential to impersonate a legitimate employee to gain access to a restricted area.

2. What misuses does this represent?

Ans: This represents a few specific "Misuse Cases":

- The act of duplicating a legitimate security token.
- Capturing a valid signal (the keycard data) and "replaying" it later to fool the reader.
- Attempting entry multiple times, perhaps at different doors or readers, to find a weakness in the system's logic.

3. Which state should activate after repeated failures?

Ans: The system should transition into an Alarm/Lockdown State. Specifically:

- The specific card reader disables itself for a set period.
- The system flags that specific (cloned) UID as "Compromised" and disables it globally.
- An automated notification is sent to the Security Operations Center (SOC) or building security.

4. What non-functional requirement is involved?

Ans: The primary non-functional requirement is Security. More specifically, it touches on:

- **Integrity:** Ensuring only valid, unaltered credentials work.
- **Availability:** Ensuring the system stays functional for others while blocking the attacker.
- **Robustness:** The system's ability to handle "negative" inputs (the wrong/cloned card) without failing or accidentally opening.

5. How can logging reduce this risk?

Ans: Logging doesn't physically stop the door from opening, but it is vital for Detection and Response:

- Logs can show that an employee "badged in" at the front door and, two minutes later, tried to "badge in" at a door five miles away—a clear sign of a cloned card (this is often called "Impossible Travel").
- If a theft occurs, logs provide a timestamped trail of which card was used, allowing security to trace the cloned card back to the original employee who was skimmed.
- If the logs show 10 failed attempts in 60 seconds, it triggers a real-time alert for a guard to intercept the intruder.

Question 04

A developer misunderstands system requirements and writes incorrect authentication logic.

1. Is this an error, defect, or bug?

Ans: This is an Error. In formal terms, an Error is the human action (the misunderstanding) that happens in the developer's mind. Once that misunderstanding is typed into the source code, it becomes a Defect (or Bug).

- **The Error:** "I thought the requirement meant *anyone* with a link can log in."
- **The Defect:** The actual lines of code that skip the password check.

2. At which SDLC phase did the issue originate?

Ans: The issue originated in the **Requirements Analysis** or **Design Phase**. Even though the developer wrote the bad code during the *Implementation (Coding)* phase, the "root cause" was a failure to understand the requirements earlier. If the requirements were unclear or the developer didn't attend the briefing, the seed was sown before a single line of code was written.

3. How could SDLC prevent this?

Ans: The SDLC isn't just a list of steps; it's a safety net. Here's how it catches these slips:

- **Requirement Validation:** Using "Walkthroughs" or "Inspections" where developers and stakeholders sit together to confirm they are on the same page.
- **Design Reviews:** Having a Senior Developer or Architect review the logic *before* coding starts.
- **Test-Driven Development (TDD):** Writing the test cases (e.g., "System should reject empty passwords") before writing the code. If the code is wrong, the test fails immediately.
- **Peer Code Reviews:** Another student or developer looking at the code and saying, "Hey, this doesn't match what the client asked for."

4. What risk does this create?

Ans: Incorrect authentication logic creates a **Critical Security Risk**, specifically:

- **Broken Authentication:** This is often at the top of the **OWASP Top 10** list. It could allow attackers to bypass login screens, hijack sessions, or gain administrative privileges.

- **Data Breach:** If the logic is flawed, unauthorized users could access sensitive user data (PII), leading to legal trouble and loss of user trust.
- **System Integrity Loss:** An attacker could change system settings or delete databases because the "gatekeeper" (the auth logic) failed to do its job.

THE END
